INSTITUTE OF COMPUTER SCIENCE AND DIGITAL INNOVATION

ACADEMIC SESSION: September 2025

BIC2234/CC206/CD215: Multimedia Programming

ASSIGNMENT **DEADLINE: 21 December 2025, 5:00 PM**

INSTRUCTIONS TO CANDIDATES

1. This assignment will contribute **30%** to your final grade.
2. This coursework is a group assignment. You may have 3-4 members in one group.
3. You are required to submit the codes and screenshots of all the possible results.

IMPORTANT

Plagiarism is not allowed, and any student found to commit such an act, 0 mark will be awarded.

Coursework must be submitted on their due dates. If a coursework is submitted after its due date, the following penalty will be imposed:

- **ONE day late : 3 marks deducted from the total marks awarded.**
- **TWO days late : 5 marks deducted from the total marks awarded.**
- **THREE or more days late : Assignment will not be marked and 0% will be awarded.**

***For example**: A student scores 10 marks for an assignment that has a total mark of 15. If the assignment is submitted one day late, the marks awarded will be 7 marks.*

You are also required to attach the screenshot of the results from the command prompt console.

INSTITUTE OF COMPUTER SCIENCE AND DIGITAL INNOVATION

GROUP ASSIGNMENT

[SEPTEMBER 2025]

CD215 - MULTIMEDIA PROGRAMMING

Lecturer: Asst. Prof. Dr. Fatima Nadeem Salem Al-Aswadi

NO.	NAME	STUDENT ID
1.	LAI KWANG ZHE	1002164155
4.	WAI YAN HTET	1002473399
5.	LIEW MAY YEE	1002473722

Deadline: 21st December 2025

Table of Content

Introduction 3
Objectives 3
Storyline 4
Code Implementation 4
Results 22
Challenges Faced and Solutions 26
Conclusion 27

Introduction

The goal of this assignment to complete the Multimedia Programming course (CD215) is to design and create an interactive application in C++ and using graphics.h library. Multimedia programming demands the need to bridge the gap between computation and representation, so that one can create interesting user experiences. In this project, we as a group have come up with a 2D arcade style game called Car Racing: Journey to Champion.

The project will show the ability to work with graphical primitives and user input and realtime game loops. The game is designed to follow a car racing theme where the player will be expected to avoid the traffic and around the obstacles in order to get to the finish line. The program makes use of graphics.h library in rendering graphics and windows.h library in handling system level functions like sound effects and timing.

There are 11 different levels in the game that presentively challenge the player with increasing difficulty, confronting them with constant challenges in the form of immobile obstacles and winding paths, and dynamic weather patterns such as rain and snow. Some of the main characteristics are a health system, nitro speed boost mechanic when the game is on the high levels, and collision detection algorithms that help to play the game fairly. The report describes the application storyline, code structure, and results of the application.

Objectives

The objective of this project is to successfully create and design a car racing game titled Car Racing: Journey to Champion that delivers an interactive gameplay across multiple levels with minimum to no bugs. The game aims to provide players with a complete experience by featuring an introduction page, main menu, level selection feature and game over page. Players are able to control a car using the keyboard to navigate through each level while also avoiding various obstacles. To better immerse players, the Car Racing: Journey to Champion is equipped with sound effects and smooth gameplay mechanics, allowing players to retry levels after failure until they successfully progress to the next level.

Storyline

The Car Racing Game follows a beginner racer who is ambitious to climb the ladder and become the Grand Pak Champion. The adventure starts with the player being introduced to his/her first basic yet efficient racing car, where s/he learns the basics of driving e.g. how to control the speed, to braking and to steering over plain roads. As the player gains confidence, he/she progresses into the busy urban settings, driving in the traffic, small streets and even the dark nights where the player must have fast decisions and reflexes. The difficulties are further enhanced on the open highways, in which the speed limitation of racing and accurate overtaking are essential in the process of keeping the control in large vehicles.

The environments of racing also get more extreme and challenging as the game goes on. The player is to overcome desert routes full of sandstorms and unstable grounds, then the mountain road which is very dangerous and needs professional skills in drifting to negotiate acute turns and sharp cliffs. The weather makes it even more challenging, as collapsing roads that result in low traction and wet weather causing slippery roads bring about unexpected challenges that challenge speed of reaction and bravery. Such phases are expected to challenge the player beyond the simple driving skills and train sophisticated racing skills.

The last parts of the game have the most challenging conditions, such as icy snowfields, on which it is hard to gain control, because of the slippery surfaces and blizzard, and volcanic tracks, which are full of lava, falling rocks, and intense heat. These degrees are the highest trial of persistence, concentration and expertise. The trip ends at the Champion Grand Prix where the player will be racing against the world fastest cars with the top racing cars. Having surmounted all the obstacles and earning the prize of winning the race, the player demonstrates that perseverance, skills, and determination can make a rookie racer a real racing legend.

Code Implementation

```
#include <graphics.h>
#include <conio.h>
#include <windows.h>
#include <iostream>
#include <vector>
#include <string>
#include <limits>
#include <cmath>
#include <cstdlib>
```

```
#include <cstdio>
#include <algorithm>
using namespace std;

int screenW = 800, screenH =
600; int carX = 400, carY = 500;
bool nitro = false; int
baseSpeed = 5; int speed =
baseSpeed; int maxHealth; int
health = maxHealth;

bool levelCleared = false;
int currentLevel = 1; bool
inLevel = false; bool
gameRunning = true; int
finishY = 50; const int
perLevelLives = 2; int aiY
= -100; static int bossY =
100; static int rockY = -
50;

//------------------------------------------------damage
taken void safeDamage(int dmg, DWORD &lastHit, DWORD cooldownMs =
400) { if (GetTickCount() - lastHit > cooldownMs) { lastHit =
GetTickCount(); health -= dmg; if (health < 0) health = 0;
Beep(600, 120);
    }
}
//-------------------------------------------------------------void drawCar(int x, int y);
void controlCar(); bool rectIntersectsRect(int l1, int t1, int r1, int b1, int l2, int
t2, int r2, int b2); bool rectCircleCollision(int rx1, int ry1, int rx2, int ry2, int cx,
int cy, int cr); void resetPlayer(); void drawHUD(); void playBeepCollision(); void
playBeepLevelUp(); void showMenu(); int showGameOver(); void showEnding(); void
showIntro(); void waitForKey(); void getCarRect(int &l, int &t, int &r, int &b); //-----
-----------------------------------------------------bool       playLevel1();       bool
playLevel2(); bool playLevel3(); bool playLevel4(); bool playLevel5(); bool playLevel6();
bool playLevel7(); bool playLevel8(); bool playLevel9(); bool playLevel10(); bool
playLevel11(); bool runLevel(int lvlNumber);
//-----------------------------------------------------------structure car
void drawCar(int x, int y) {
//main body of car
    setcolor(WHITE);
    setfillstyle(SOLID_FILL,
    RED);
    rectangle(x - 30, y - 20, x + 30, y);
    floodfill(x, y - 10, WHITE); rectangle(x -
    20, y - 40, x + 20, y - 20); floodfill(x,
    y - 30, WHITE);

//windows of car setcolor(LIGHTCYAN);
    setfillstyle(SOLID_FILL,
    LIGHTCYAN);
    rectangle(x - 18, y - 38, x, y - 22);
    floodfill(x - 10, y - 30, LIGHTCYAN);
    rectangle(x, y - 38, x + 18, y - 22);
    floodfill(x + 10, y - 30, LIGHTCYAN);
```

```
        //tyres of car
    setcolor(WHITE);
    setfillstyle(SOLID_FILL, BLACK);
    circle(x - 18, y + 5, 8);
    floodfill(x - 18, y + 5, WHITE);
    circle(x + 18, y + 5, 8);
    floodfill(x + 18, y + 5, WHITE);
        //rear end of the car
    setcolor(YELLOW);
    setfillstyle(SOLID_FILL, YELLOW);
    rectangle(x + 30, y - 15, x + 35, y - 5);
    floodfill(x + 32, y - 10, YELLOW);
        //front end of the car
    setcolor(LIGHTRED);
    setfillstyle(SOLID_FILL, LIGHTRED);
    rectangle(x - 35, y - 15, x - 30, y - 5);
    floodfill(x - 32, y - 10, LIGHTRED);
}
//-------------------------------------------------nitro usage
void controlCar() {
    if (currentLevel >= 4) { if
        (GetAsyncKeyState('N') & 0x8000) {
            nitro = true; speed =
            baseSpeed * 2;
        } else { nitro =
            false; speed
            = baseSpeed;
        }
    } else { nitro =
        false; speed =
        baseSpeed;
    } if (GetAsyncKeyState(VK_LEFT) & 0x8000) carX -=
    speed; if (GetAsyncKeyState(VK_RIGHT) & 0x8000) carX
    += speed; if (GetAsyncKeyState(VK_UP)   & 0x8000) carY
    -= speed; if (GetAsyncKeyState(VK_DOWN) & 0x8000) carY
    += speed;
        //restricts car's movement if (carX <
    35) carX = 35; if (carX > screenW - 35) carX
    = screenW - 35; if (carY < 10) carY = 10; if
    (carY > screenH - 10) carY = screenH - 10;
}
//---------------------------------detects collision with rectangular
obstacles bool rectIntersectsRect(int l1, int t1, int r1, int b1, int l2, int
t2, int r2, int b2) {
    return !(r1 < l2 || r2 < l1 || b1 < t2 || b2 < t1);
}
//detects collision with circular obstacles bool rectCircleCollision(int rx1, int
ry1, int rx2, int ry2, int cx, int cy, int cr) { int closestX = std::max(rx1,
std::min(cx, rx2)); int closestY = std::max(ry1, std::min(cy, ry2)); int dx = cx -
closestX; int dy = cy - closestY; return (dx * dx + dy * dy) <= (cr * cr);
}
//-----------------------------------------------reset lives per
level void resetPlayer() { carX = screenW / 2; carY = screenH - 100;
maxHealth = perLevelLives; health = maxHealth; nitro = false; speed
= baseSpeed;
//-------------------------------------------------------
void drawHUD() { if (health < 0) health = 0; if (health >
```

```
maxHealth) health = maxHealth; setcolor(WHITE);
outtextxy(10, 10, (char*)"Health:");
        //health bar int hbX = 120, hbY
    = 10; int cellW = 18; for (int i = 0;
    i < maxHealth; ++i) { if (i < health)
    {
            setfillstyle(SOLID_FILL, GREEN); bar(hbX + i
            * (cellW + 4), hbY, hbX + i * (cellW + 4) +
            cellW, hbY + 16);
        } else {
            setcolor(WHITE); rectangle(hbX + i * (cellW + 4),
            hbY, hbX + i * (cellW + 4) + cellW, hbY + 16);
        }
    }
         //nitro state if (currentLevel >= 4) {
    outtextxy(10, 35, (char*)"Nitro:(Hold N)"); if
    (nitro) { outtextxy(120, 50, (char*)"ON");
        } else {
            outtextxy(120, 50,(char*)"OFF");
        }
    }
         //shows current level char
    lvlStr[32]; sprintf(lvlStr, "Level %d",
    currentLevel); outtextxy(screenW - 150,
    10, lvlStr);
}
//-----------------------------------sound effect of
collision void playBeepCollision() { Beep(600, 120);
}
//sound effect of finishing a level
void playBeepLevelUp() {
    Beep(1000, 120);
    Beep(1300, 120);
}
//--------------------------------determine the location of car that triggers a
collision void getCarRect(int &l, int &t, int &r, int &b) { l = carX - 30; t = carY -
40; r = carX + 30; b = carY + 5;
}
//-----------------------------------------------level
1 bool playLevel1() { cleardevice();

    settextstyle(DEFAULT_FONT, HORIZ_DIR, 2);
    char title[] = "Car Drive Testing"; int
    xPos = screenW - textwidth(title) - 10;
    outtextxy(xPos, 60, title);
         //structure of tracks int trackW = 300; int
    laneL = screenW/2 - trackW/2; int laneR = screenW/2
    + trackW/2; setcolor(LIGHTGRAY); rectangle(laneL, 0,
    laneR, screenH); //structure of finish line int
    sqSize = 20; for (int row = 0; row < 2; row++) { for
    (int x = laneL; x < laneR; x += sqSize) { if (((x -
    laneL) / sqSize + row) % 2 == 0) {
    setfillstyle(SOLID_FILL, WHITE);
            } else {
                setfillstyle(SOLID_FILL, BLACK);
            } bar(x, finishY + (row * sqSize), x + sqSize, finishY + (row * sqSize) +
            sqSize);
```

```
        }
    }
         //car's collision area
    int l, t, r, b;
    getCarRect(l, t, r, b);
         //damage taken in collision with side tracks
    static DWORD lastHit = 0; if (l < laneL || r >
    laneR) {
        safeDamage(1, lastHit, 500);
    } controlCar();
    drawCar(carX, carY);
    drawHUD();
         //if car passes finish line, level is completed
    if (carY < finishY) return true;
    delay(20);
    return false;
}
//----------------------------------------level
2 bool playLevel2() { cleardevice();

    settextstyle(DEFAULT_FONT, HORIZ_DIR, 2); char
    title[] = "City Cruiser - Curvy Roads"; int xPos =
    screenW - textwidth(title) - 20; outtextxy(xPos, 60,
    title); int roadHalfWidth = 140; int roadL =
    (screenW / 2) - roadHalfWidth; int roadR = (screenW
    / 2) + roadHalfWidth; setcolor(WHITE);
    rectangle(roadL, 0, roadR, screenH);
    setcolor(LIGHTGRAY); rectangle(roadL - 20, 0, roadL,
    screenH); rectangle(roadR, 0, roadR + 20, screenH);
    int sqSize = 20; for (int row = 0; row < 2; row++) {
    for (int x = roadL; x < roadR; x += sqSize) { if
    (((x - roadL) / sqSize + row) % 2 == 0) {
    setfillstyle(SOLID_FILL, WHITE);
            } else {
                setfillstyle(SOLID_FILL, BLACK);
            } bar(x, finishY + (row * sqSize), x + sqSize, finishY + (row * sqSize) + sqSize);
        }

   //circular obstacles int radius = 35; int c1x
    = roadL + 50, c1y = screenH / 2 + 60; int
    c2x = roadR - 50, c2y = screenH / 3 + 80;
    int c3x = screenW / 2, c3y = 150;
    setcolor(YELLOW); setfillstyle(SOLID_FILL, YELLOW);
    circle(c1x, c1y, radius); floodfill(c1x, c1y, YELLOW);
    circle(c2x, c2y, radius); floodfill(c2x, c2y, YELLOW);
    circle(c3x, c3y, radius); floodfill(c3x, c3y, YELLOW);
    int l, t, r, b; getCarRect(l, t, r, b); bool hit =
    false; if (l < roadL || r > roadR) hit = true;
    //checks collision with circular obstacles if
    (rectCircleCollision(l, t, r, b, c1x, c1y, radius) ||
    rectCircleCollision(l, t, r, b, c2x, c2y, radius) ||
    rectCircleCollision(l, t, r, b, c3x, c3y, radius)) { hit
    = true; }
         //damage taken upon collision static
    DWORD lastHit = 0; if (hit && GetTickCount()
    - lastHit > 500) { lastHit = GetTickCount();
    health--; playBeepCollision();
```

```
    } controlCar();
    drawCar(carX, carY);
    drawHUD(); if (carY
    < finishY) return
    true;
    delay(20);
    return false;
}

//----------------------------------------level
3 bool playLevel3() { cleardevice();

    settextstyle(DEFAULT_FONT, HORIZ_DIR, 2); char
    title[] = "Night Rider - Moving Obstacles"; int
    xPos = screenW - textwidth(title) - 20;
    outtextxy(xPos, 60, title); int roadHalfWidth =
    120; int roadL = (screenW / 2) - roadHalfWidth;
    int roadR = (screenW / 2) + roadHalfWidth;
    setcolor(WHITE); rectangle(roadL, 0, roadR,
    screenH); int sqSize = 20; for (int row = 0; row
    < 2; row++) { for (int x = roadL; x < roadR; x +=
    sqSize) {

            if (((x - roadL) / sqSize + row) % 2 == 0) {
                setfillstyle(SOLID_FILL, WHITE);
            } else {
                setfillstyle(SOLID_FILL, BLACK);
            } bar(x, finishY + (row * sqSize), x + sqSize, finishY + (row * sqSize) +
            sqSize);
        }
    }
         //moving circular obstacles const int
    count = 3; const int baseY = screenH / 2;
    const int spacing = 120; const int radius =
    26; int laneCenter = screenW / 2; int
    maxMove = roadHalfWidth - radius - 10;
    const double speeds[3] = { 1.0, 1.3, 0.8 };
    const double phases[3] = { 0.0, 2.0, 4.0 };
    double timeSec = GetTickCount() / 1000.0;
    for (int i = 0; i < count; ++i) { int y =
    baseY - spacing + i * spacing; int x =
    laneCenter + (int)(sin(timeSec * speeds[i]
    + phases[i]) * maxMove);
    setcolor(LIGHTRED);
    setfillstyle(SOLID_FILL, LIGHTRED);
    circle(x, y, radius); floodfill(x, y,
    LIGHTRED); if (rectCircleCollision(l, t, r,
    b, x, y, radius)) { hit = true;
        }
    }
    int l, t, r, b; getCarRect(l,
    t, r, b); bool hit = false;
    static DWORD lastHit = 0; if
    (l < roadL || r > roadR) {
    hit = true;
```

```
    } if (hit && GetTickCount() - lastHit > 500)
    { lastHit = GetTickCount(); health--;
    playBeepCollision();
    } controlCar();
    drawCar(carX, carY);
    drawHUD(); if (carY
    < finishY) return
    true;
    delay(20);
    return false;
}

//-----------------------------------------level
4 bool playLevel4() { cleardevice();

    settextstyle(DEFAULT_FONT, HORIZ_DIR, 2);
    char title[] = "Highway Sprint - Nitro Test";
    int xPos = screenW - textwidth(title) - 20;
    outtextxy(xPos, 60, title); int roadHalfWidth
    = 140; int roadL = (screenW / 2) -
    roadHalfWidth; int roadR = (screenW / 2) +
    roadHalfWidth; setcolor(WHITE);
    rectangle(roadL, 0, roadR, screenH);
    setcolor(LIGHTGRAY); rectangle(roadL - 20, 0,
    roadL, screenH); rectangle(roadR, 0, roadR +
    20, screenH); int sqSize = 20; for (int row =
    0; row < 2; row++) {
        for (int x = roadL; x < roadR; x += sqSize) {

            if (((x - roadL) / sqSize + row) % 2 == 0) {
                setfillstyle(SOLID_FILL, WHITE);
            } else {
                setfillstyle(SOLID_FILL, BLACK);
            }
            bar(x, finishY + (row * sqSize), x + sqSize, finishY + (row * sqSize) + sqSize);
        }
    }
    int l, t, r, b; getCarRect(l,
    t, r, b); bool hitLane =
    false; if (l < roadL || r >
    roadR) { hitLane = true;
    } static DWORD lastHit = 0; if (hitLane &&
    GetTickCount() - lastHit > 500) { lastHit =
    GetTickCount(); health--; playBeepCollision();
    } controlCar();
    drawCar(carX, carY);
    drawHUD(); if (carY
    < finishY) return
    true;
    delay(20);
    return false;
}

//-----------------------------------------level
5 bool playLevel5() { cleardevice();
```

```
    settextstyle(DEFAULT_FONT, HORIZ_DIR, 2); char
    title[] = "Traffic Run - Avoid Moving Cars"; int
    xPos = screenW - textwidth(title) - 20;
    outtextxy(xPos, 60, title); int roadHalfWidth = 120;
    int roadL = (screenW / 2) - roadHalfWidth; int roadR
    = (screenW / 2) + roadHalfWidth; setcolor(WHITE);
    rectangle(roadL, 0, roadR, screenH); int sqSize =
    20; for (int row = 0; row < 2; row++) { for (int x =
    roadL; x < roadR; x += sqSize) { if (((x - roadL) /
    sqSize + row) % 2 == 0) { setfillstyle(SOLID_FILL,
    WHITE);
            } else {
                setfillstyle(SOLID_FILL, BLACK);
            } bar(x, finishY + (row * sqSize), x + sqSize, finishY + (row * sqSize) +
            sqSize);
        }
    }
         //moving cars obstacles static
    int ai1Y = -80; static int ai2Y = -
    300; ai1Y += 6; ai2Y += 8; if (ai1Y >
    screenH + 60) ai1Y = -100; if (ai2Y >
    screenH + 60) ai2Y = -200; int aiW =
    40; int aiH = 60;
    int ai1X = (roadL + roadR) / 2 - 50;
    int ai2X = (roadL + roadR) / 2 + 50;
    setcolor(GREEN); setfillstyle(SOLID_FILL, GREEN);
    rectangle(ai1X - aiW/2, ai1Y - aiH, ai1X + aiW/2,
    ai1Y); floodfill(ai1X, ai1Y - aiH/2, GREEN);
    setcolor(MAGENTA); setfillstyle(SOLID_FILL, MAGENTA);
    rectangle(ai2X - aiW/2, ai2Y - aiH, ai2X + aiW/2,
    ai2Y); floodfill(ai2X, ai2Y - aiH/2, MAGENTA);

    int l, t, r, b; getCarRect(l,
    t, r, b); bool hit = false;
    if (l < roadL || r > roadR) {
        hit = true; } if (rectIntersectsRect(l, t, r,
    b, ai1X - aiW/2, ai1Y - aiH, ai1X + aiW/2, ai1Y))
    { hit = true; } if (rectIntersectsRect(l, t, r,
    b, ai2X - aiW/2, ai2Y - aiH, ai2X + aiW/2, ai2Y))
    { hit = true; } static DWORD lastHit = 0; if (hit
    && GetTickCount() - lastHit > 500) {
        lastHit = GetTickCount();
        health--;
        playBeepCollision();
    } controlCar();
    drawCar(carX, carY);
    drawHUD(); if (carY
    < finishY) return
    true;
    delay(20);
    return false;
}

//---------------------------------level 6
bool playLevel6() {
    cleardevice();
```

```
    settextstyle(DEFAULT_FONT, HORIZ_DIR, 2);
    char title[] = "Rain Rush - Slippery Road";
    int xPos = screenW - textwidth(title) - 20;
    outtextxy(xPos, 60, title); int
    laneHalfWidth = 120; int laneL =
    (screenW/2) - laneHalfWidth; int laneR =
    (screenW/2) + laneHalfWidth;
    setcolor(WHITE); rectangle(laneL, 0, laneR,
    screenH); int sqSize = 20; for (int row =
    0; row < 2; row++) {
        for (int x = laneL; x < laneR; x += sqSize) { if
            (((x - laneL) / sqSize + row) % 2 == 0) {
            setfillstyle(SOLID_FILL, WHITE);
            } else {
                setfillstyle(SOLID_FILL, BLACK);
            } bar(x, finishY + (row * sqSize), x + sqSize, finishY + (row * sqSize) +
            sqSize);
        }
    }
         //simulate rain
    setcolor(LIGHTBLUE);
    for (int i = 0; i < 80; i++) {
        int x = rand() % screenW;
        int y = rand() % screenH;
        line(x, y, x + 2, y + 10);
    }
         //define moving rectangular obstacle static int barrierY = -40; barrierY +=
    6; if (barrierY > screenH + 40) barrierY = -60; int bw = 80; int bx1 = screenW/2 -
    bw/2; int bx2 = screenW/2 + bw/2; setcolor(RED); bar(bx1, barrierY, bx2, barrierY
    + 20); int l, t, r, b; getCarRect(l, t, r, b); static DWORD lastHit = 0; bool hit
    = false; if (l < laneL || r > laneR) hit = true; if (rectIntersectsRect(l, t, r,
    b, bx1, barrierY, bx2, barrierY + 20)) hit = true; if (hit) safeDamage(1,
    lastHit); controlCar(); drawCar(carX, carY); drawHUD(); if (carY < finishY) return
    true;
         delay(20);
    return false;
}

//------------------------------level
7 bool playLevel7() { cleardevice();

    settextstyle(DEFAULT_FONT, HORIZ_DIR, 2);
    char title[] = "Mountain Pass - Narrow Road";
    int xPos = screenW - textwidth(title) - 20;
    outtextxy(xPos, 60, title); int roadHalfWidth
    = 100; int roadL = (screenW / 2) -
    roadHalfWidth; int roadR = (screenW / 2) +
    roadHalfWidth; setcolor(WHITE);
    rectangle(roadL, 0, roadR, screenH);
    setcolor(LIGHTGRAY); rectangle(roadL - 20, 0,
    roadL, screenH); rectangle(roadR, 0, roadR +
    20, screenH);
    int sqSize = 20; for (int row = 0; row < 2; row++) {
    for (int x = roadL; x < roadR; x += sqSize) { if
    (((x - roadL) / sqSize + row) % 2 == 0) {
    setfillstyle(SOLID_FILL, WHITE);
            } else {
```

```
                setfillstyle(SOLID_FILL, BLACK);
            } bar(x, finishY + (row * sqSize), x + sqSize, finishY + (row * sqSize) +
            sqSize);
        }
    }
         //moving rock obstacle static int
    rockY = -60; rockY += 7; if (rockY >
    screenH + 60) rockY = -80; int rockX =
    screenW / 2; int rockR = 20;
    setcolor(DARKGRAY);
    setfillstyle(SOLID_FILL, DARKGRAY);
    circle(rockX, rockY, rockR);
    floodfill(rockX, rockY, DARKGRAY);
    int l, t, r, b; getCarRect(l, t, r,
    b); bool hit = false; if (l < roadL
    || r > roadR) { hit = true;
    } if (rectCircleCollision(l, t, r, b, rockX, rockY, rockR))
    { hit = true;
    } static DWORD lastHit = 0; if (hit &&
    GetTickCount() - lastHit > 500) { lastHit =
    GetTickCount(); health--;
    playBeepCollision();
    } controlCar(); drawCar(carX,
    carY); drawHUD(); if (carY <
    finishY) return true; delay(20);
    return false;
}

//-----------------------------------level
8 bool playLevel8() { cleardevice();

    settextstyle(DEFAULT_FONT, HORIZ_DIR, 2);
    char title[] = "Tunnel Run - Low Visibility";
    int xPos = screenW - textwidth(title) - 20;
    outtextxy(xPos, 60, title); int roadHalfWidth
    = 140; int roadL = (screenW / 2) -
    roadHalfWidth; int roadR = (screenW / 2) +
    roadHalfWidth;
    setcolor(DARKGRAY);
    rectangle(roadL, 0, roadR, screenH);
    setcolor(LIGHTGRAY); rectangle(roadL - 25, 0, roadL,
    screenH); rectangle(roadR, 0, roadR + 25, screenH);
    int sqSize = 20; for (int row = 0; row < 2; row++) {
    for (int x = roadL; x < roadR; x += sqSize) { if
    (((x - roadL) / sqSize + row) % 2 == 0) {
    setfillstyle(SOLID_FILL, WHITE);
            } else {
                setfillstyle(SOLID_FILL, BLACK);
            } bar(x, finishY + (row * sqSize), x + sqSize, finishY + (row * sqSize) +
            sqSize);
        }
    }
         //moving tunnel obstacle
    static int obsX = roadL + 20;
    static int obsY = -60; static
    int dir = 1;
```

```
    obsY += 6; if (obsY > screenH + 60) obsY = -80;
    obsX += dir * 4; if (obsX < roadL + 20 || obsX >
    roadR - 60) dir *= -1; int obsW = 60; int obsH =
    20; setcolor(RED); setfillstyle(SOLID_FILL,
    RED); rectangle(obsX, obsY, obsX + obsW, obsY +
    obsH); floodfill(obsX + 5, obsY + 5, RED); int
    l, t, r, b; getCarRect(l, t, r, b); bool hit =
    false; if (l < roadL || r > roadR) { hit = true;
    } if (rectIntersectsRect(l, t, r, b, obsX, obsY, obsX + obsW,
    obsY + obsH)) {
        hit = true;
    } static DWORD lastHit = 0; if (hit &&
    GetTickCount() - lastHit > 500) { lastHit =
    GetTickCount(); health--;
    playBeepCollision();
    } controlCar();
    drawCar(carX, carY);
    drawHUD(); if (carY
    < finishY) return
    true;
    delay(20);
    return false;
}

//-------------------------------------level 9
bool playLevel9() {
    cleardevice();

    settextstyle(DEFAULT_FONT, HORIZ_DIR, 2); char
    title[] = "Highway Dodge - Falling Blocks"; int xPos
    = screenW - textwidth(title) - 20; outtextxy(xPos,
    60, title); int roadHalfWidth = 130; int roadL =
    (screenW / 2) - roadHalfWidth; int roadR = (screenW
    / 2) + roadHalfWidth; setcolor(WHITE);
    rectangle(roadL, 0, roadR, screenH); int sqSize =
    20; for (int row = 0; row < 2; row++) { for (int x =
    roadL; x < roadR; x += sqSize) { if (((x - roadL) /
    sqSize + row) % 2 == 0) { setfillstyle(SOLID_FILL,
    WHITE);
            } else {
                setfillstyle(SOLID_FILL, BLACK);
            }

            bar(x, finishY + (row * sqSize), x + sqSize, finishY + (row * sqSize) + sqSize);
        }
    } const int blockCount = 3; static int
    blockY[blockCount] = { -100, -300, -500 }; static int
    blockX[blockCount] = { screenW/2 - 60, screenW/2 +
    20, screenW/2 - 20
    }; int blockW = 50; int
    blockH = 25; int l, t, r,
    b; getCarRect(l, t, r,
    b); static DWORD lastHit
    = 0;
         //falling blocks obstacles for (int i = 0; i < blockCount;
    i++) { blockY[i] += 6; if (blockY[i] > screenH + 40) {
```

```
    blockY[i] = -100; blockX[i] = roadL + 20 + rand() % (roadR -
    roadL - 60);
        } setcolor(RED); setfillstyle(SOLID_FILL, RED);
        rectangle(blockX[i], blockY[i], blockX[i] +
        blockW, blockY[i] + blockH);
        floodfill(blockX[i] + 5, blockY[i] + 5,
        RED); if (rectIntersectsRect( l, t, r, b,
                blockX[i], blockY[i], blockX[i] + blockW,
                blockY[i] + blockH)) {
            safeDamage(1, lastHit, 500);
        }
    }
    if (l < roadL || r > roadR) {
        safeDamage(1, lastHit, 600);
    } controlCar();
    drawCar(carX, carY);
    drawHUD(); if (carY
    < finishY) return
    true;
    delay(20);
    return false;
}

//-----------------------------------level
10 bool playLevel10() { cleardevice();

    settextstyle(DEFAULT_FONT, HORIZ_DIR, 2); char
    title[] = "Lava Dodge - Avoid the Lava"; int xPos =
    screenW - textwidth(title) - 20; outtextxy(xPos, 60,
    title); int roadHalfWidth = 140; int roadL =
    (screenW / 2) - roadHalfWidth; int roadR = (screenW
    / 2) + roadHalfWidth; setcolor(DARKGRAY);
    rectangle(roadL, 0, roadR, screenH);
    setcolor(LIGHTGRAY); rectangle(roadL - 25, 0, roadL,
    screenH); rectangle(roadR, 0, roadR + 25, screenH);
    int sqSize = 20; for (int row = 0; row < 2; row++) {
    for (int x = roadL; x < roadR; x += sqSize) { if
    (((x - roadL) / sqSize + row) % 2 == 0) {
    setfillstyle(SOLID_FILL, WHITE);
            } else {
                setfillstyle(SOLID_FILL, BLACK);
            } bar(x, finishY + (row * sqSize), x + sqSize, finishY + (row * sqSize) +
            sqSize);
        }
    }
         //lava obstacles
    static int lavaX[4] = {
    screenW/2 - 80,
    screenW/2 + 60,
    screenW/2, screenW/2 -
    20
    }; static int lavaY[4] = { -50, -250, -450, -650
    }; int lavaR = 18; int l, t, r, b; getCarRect(l,
    t, r, b); static DWORD lastHit = 0;
```

```
    for (int i = 0; i < 4; i++) { lavaY[i] += 6 + i; if (lavaY[i]
        > screenH + 40) { lavaY[i] = -100; lavaX[i] = roadL + 30 +
        rand() % (roadR - roadL - 60);
        }
        setcolor(RED);
        setfillstyle(S
        OLID_FILL,
        RED);
        circle(lavaX[i
        ], lavaY[i],
        lavaR);
        floodfill(lava
        X[i],
        lavaY[i],
        RED); if
        (rectCircleCol
        lision(l, t,
        r, b,
        lavaX[i],
        lavaY[i],
        lavaR)) {
            safeDamage(1, lastHit, 400);
        }
    }
    if (l < roadL || r > roadR) {
        safeDamage(1, lastHit, 500);
    } controlCar();
    drawCar(carX, carY);
    drawHUD(); if (carY
    < finishY) return
    true; delay(20);
    return false;
}
//-------------------------------------level
11 bool playLevel11() { cleardevice();

    settextstyle(DEFAULT_FONT, HORIZ_DIR, 2); char
    title[] = "Snow Run - Slippery Finish"; int xPos =
    screenW - textwidth(title) - 20; outtextxy(xPos, 60,
    title); int roadHalfWidth = 130; int roadL =
    (screenW / 2) - roadHalfWidth; int roadR = (screenW
    / 2) + roadHalfWidth; setcolor(WHITE);
    rectangle(roadL, 0, roadR, screenH);
    setcolor(LIGHTGRAY); rectangle(roadL - 20, 0, roadL,
    screenH); rectangle(roadR, 0, roadR + 20, screenH);
    int sqSize = 20; for (int row = 0; row < 2; row++) {
    for (int x = roadL; x < roadR; x += sqSize) { if
    (((x - roadL) / sqSize + row) % 2 == 0) {
    setfillstyle(SOLID_FILL, WHITE);
            } else {
                setfillstyle(SOLID_FILL, BLACK);
            } bar(x, finishY + (row * sqSize), x + sqSize, finishY + (row * sqSize) +
            sqSize);
        }
    }
         //snow effect
    setcolor(WHITE);
```

```
    for (int i = 0; i < 120; i++) {
        int sx = rand() % screenW;
        int sy = rand() % screenH;
        circle(sx, sy, 1);
    }
         //ice obstacle
    static int iceY1 = -80;
    static int iceY2 = -300;
    iceY1 += 6;
    iceY2 += 8; if (iceY1 > screenH + 60)
    iceY1 = -100; if (iceY2 > screenH + 60)
    iceY2 = -200; int iceW = 60, iceH = 20;
    int iceX1 = roadL + 40; int iceX2 =
    roadR - 100;

    setcolor(LIGHTCYAN); setfillstyle(SOLID_FILL, LIGHTCYAN); rectangle(iceX1,
    iceY1, iceX1 + iceW, iceY1 + iceH); floodfill(iceX1 + 5, iceY1 + 5,
    LIGHTCYAN); rectangle(iceX2, iceY2, iceX2 + iceW, iceY2 + iceH);
    floodfill(iceX2 + 5, iceY2 + 5, LIGHTCYAN); int l, t, r, b; getCarRect(l, t,
    r, b); bool hit = false; if (l < roadL || r > roadR) hit = true; if
    (rectIntersectsRect(l, t, r, b,iceX1, iceY1, iceX1 + iceW, iceY1 + iceH))
    hit = true;
    if (rectIntersectsRect(l, t, r, b, iceX2, iceY2, iceX2 + iceW, iceY2 + iceH))
        hit = true;
    static DWORD lastHit = 0; if (hit &&
    GetTickCount() - lastHit > 500) { lastHit =
    GetTickCount(); health--;
    playBeepCollision();
    } controlCar();
    drawCar(carX, carY);
    drawHUD(); if (carY
    < finishY) return
    true;
    delay(20);
    return false;
}

//---------------------------------running through
level bool runLevel(int lvlNumber) { maxHealth =
perLevelLives; resetPlayer(); currentLevel = lvlNumber;
inLevel = true; bool levelWin = false;
         //resets player's lives
    while (inLevel) {
        if (health <= 0) { int choice =
            showGameOver(); if (choice
            == 1) { resetPlayer();
            health = maxHealth;
            continue;
            } else {
                return false;
            } } switch

        (lvlNumber) { case

        1: levelWin =

        playLevel1();

        break; case 2:
```

```
        levelWin =
        playLevel2();
        break; case 3:
        levelWin =
        playLevel3();
        break; case 4:
        levelWin =
        playLevel4();
        break; case 5:
        levelWin =
        playLevel5();
        break; case 6:
        levelWin =
        playLevel6();
        break; case 7:
        levelWin =
        playLevel7();
        break; case 8:
        levelWin =
        playLevel8();
        break; case 9:
        levelWin =
        playLevel9();
        break; case 10:
        levelWin =
        playLevel10();
        break; case 11:
        levelWin =
        playLevel11();
        break; default:
        levelWin = true;
         break;
        } if (levelWin) {
        playBeepLevelUp();
        inLevel = false;
        return true;
        } if (GetAsyncKeyState(VK_ESCAPE) & 0x8000)
        { inLevel = false; return false;
        } }
    return false;
}
```

```
//-----------------------------------introduction
page void showIntro() { cleardevice();
        //borders around the screen setcolor(WHITE);
    rectangle(10, 10, screenW - 10, screenH - 10);
    setcolor(WHITE); rectangle(15, 15, screenW - 15,
    screenH - 15); char title[] = "CAR RACING: JOURNEY
    TO CHAMPION"; settextstyle(TRIPLEX_FONT, HORIZ_DIR,
    4); int titleX = (screenW / 2) - (textwidth(title) /
    2); int titleY = 180;
    setcolor(DARKGRAY); outtextxy(titleX + 4, titleY + 4, title);
    setcolor(LIGHTRED); outtextxy(titleX, titleY, title); setcolor(WHITE);
    line((screenW/2) - 200, titleY + 60, (screenW/2) + 200, titleY + 60);
    line((screenW/2) - 200, titleY + 65, (screenW/2) + 200, titleY + 65);
    settextstyle(DEFAULT_FONT, HORIZ_DIR, 2); setcolor(LIGHTCYAN); char
    ctrl[] = "Controls: Arrow keys to move | N for Nitro | ESC to exit";
    outtextxy((screenW / 2) - (textwidth(ctrl) / 2), 320, ctrl);
    settextstyle(DEFAULT_FONT, HORIZ_DIR, 2); setcolor(YELLOW); char prompt[]
    = "PRESS ANY KEY TO START YOUR ENGINE"; outtextxy((screenW / 2) -
    (textwidth(prompt) / 2), 420, prompt); getch();
}

//---------------------------------menu page
void showMenu() {
    while (true) {
    cleardevice();

        settextstyle(3, 0, 4); outtextxy( (screenW/2) - 200, 80, (char*)"MAIN MENU");
        settextstyle(DEFAULT_FONT, HORIZ_DIR, 2); outtextxy( (screenW/2) - 200, 180,
        (char*)"1. Start (Play from Level 1)"); outtextxy( (screenW/2) - 200, 230,
        (char*)"2. Level Select"); outtextxy( (screenW/2) - 200, 280, (char*)"3. Exit");
        outtextxy( (screenW/2) - 180, screenH - 120, (char*)"Press 1 / 2 / 3 to
        choose");

        if (GetAsyncKeyState('1') & 0x8000) { while
            (GetAsyncKeyState('1') & 0x8000) delay(10); int
            lvl = 1; bool allWin = true; while (lvl <= 11) {
            bool won = runLevel(lvl); if (won) { lvl++;
                } else { allWin =
                    false; break;
                }
            }
            if (lvl > 10 && allWin) { cleardevice(); setcolor(YELLOW); settextstyle(3, 0, 5);
                outtextxy((screenW/2)-200, screenH/2 - 80, (char*)"CONGRATULATIONS!");
                settextstyle(DEFAULT_FONT, HORIZ_DIR, 2); outtextxy((screenW/2)-260, screenH/2 - 20,
                (char*)"You've completed all 11 levels!"); outtextxy((screenW/2)-160, screenH/2 + 40,
                (char*)"Press Enter to return to Main Menu"); Beep(1500, 200); Beep(1800, 200);

                while (!(GetAsyncKeyState(VK_RETURN) & 0x8000))
            delay(50); while (GetAsyncKeyState(VK_RETURN) & 0x8000)
            delay(10); }

        } else if (GetAsyncKeyState('2') & 0x8000) { while
            (GetAsyncKeyState('2') & 0x8000) delay(10); int
            sel = 0; const int total = 11; bool leave =
            false; while (!leave) { cleardevice();
```

```
                settextstyle(DEFAULT_FONT, HORIZ_DIR, 2);
                outtextxy((screenW/2) - 100, 40, (char*)"LEVEL
                SELECT"); settextstyle(DEFAULT_FONT, HORIZ_DIR, 1); int
                colWidth = 250; int rowHeight = 40; int startX =
                (screenW/2) - colWidth; int startY = 120; for (int i =
                0; i < total; ++i) { int col = (i < 5) ? 0 : 1; int row
                = (i < 5) ? i : (i - 5);
                    int x = (screenW/2) - 150 + (col * colWidth);
                    int y = startY + row * rowHeight;

                    if (i == sel) {
                        setcolor(YELLOW);
                        rectangle(x - 10, y - 4, x + 160, y + 20);
                    } setcolor(WHITE); char buf[64];
                    sprintf(buf, "Level %2d", i +
                    1); outtextxy(x, y, buf);
                }

                outtextxy((screenW/2) - 240, screenH - 120,
                          (char*)"Use Up/Down or W/S to move, Enter to play");
                outtextxy((screenW/2) - 260, screenH - 100,
                          (char*)"Press 1..9 for 1-9, 0 for 10, shift+1 for 11, Esc to return"); if
                ((GetAsyncKeyState(VK_DOWN) & 0x8000) || (GetAsyncKeyState('S') & 0x8000)) { sel = (sel +
1) % total; delay(140); } if ((GetAsyncKeyState(VK_UP) & 0x8000)      || (GetAsyncKeyState('W') & 0x8000)) {
                sel = (sel - 1
+ total) % total; delay(140); } if (GetAsyncKeyState(VK_RETURN) & 0x8000)
                { while (GetAsyncKeyState(VK_RETURN) & 0x8000) delay(10);
                runLevel(sel + 1);
                } if ((GetAsyncKeyState('1') & 0x8000) && (GetAsyncKeyState(VK_SHIFT) & 0x8000)) {
                  while
((GetAsyncKeyState('1') & 0x8000) && (GetAsyncKeyState(VK_SHIFT) & 0x8000)) delay(10);
                  runLevel(11);
         }
                for (int k = 1; k <= 9; ++k) {
                    if (GetAsyncKeyState('0' + k) & 0x8000) { while
                        (GetAsyncKeyState('0' + k) & 0x8000) delay(10);
                        runLevel(k); break;
                    }
                } if (GetAsyncKeyState('0') & 0x8000) { while
                (GetAsyncKeyState('0') & 0x8000) delay(10); runLevel(10);
                                    }
                if (GetAsyncKeyState(VK_ESCAPE) & 0x8000) { while
                                    (GetAsyncKeyState(VK_ESCAPE) & 0x8000) delay(10);
                                    leave = true;
                                    }
                delay(40);
            }
        } else if (GetAsyncKeyState('3') & 0x8000) { while
    (GetAsyncKeyState('3') & 0x8000) delay(10); cleardevice();
    outtextxy(260, 260,(char*) "Exiting... Thanks for playing!");
    delay(800); gameRunning = false; break; } if (!gameRunning) break;
    delay(150); }
}
//---------------------------------game over
page int showGameOver() { cleardevice();
```

```
    setcolor(RED); settextstyle(3, 0, 4);
    outtextxy(200, 220, (char*)"GAME OVER");
    settextstyle(DEFAULT_FONT, HORIZ_DIR, 2);
    outtextxy(200, 280,(char*) "Press R to retry level, M for menu");
    Beep(400, 400); Beep(300, 300); while (true) { if
    (GetAsyncKeyState('R') & 0x8000) { while
    (GetAsyncKeyState('R') & 0x8000) delay(10); return 1; }
    if (GetAsyncKeyState('M') & 0x8000) { while
    (GetAsyncKeyState('M') & 0x8000) delay(10); return 0; }
    delay(100);
    }
}
//-------------------------------------ending
page void showEnding() { cleardevice();

    setcolor(YELLOW); settextstyle(3, 0, 5);
    outtextxy(120, 200,(char*) "YOU ARE THE CHAMPION!");
    settextstyle(DEFAULT_FONT, HORIZ_DIR, 2);
    outtextxy(160, 300, (char*)"Congratulations — you've completed all 11 levels!");
    Beep(1500, 200);
    Beep(1800, 200); Beep(2100, 200); outtextxy(180, 380,(char*)
    "Press Enter to return to menu..."); while
    (!(GetAsyncKeyState(VK_RETURN) & 0x8000)) delay(50); while
    (GetAsyncKeyState(VK_RETURN) & 0x8000) delay(10);
}
//-------------------------------------
void waitForKey() { while (!kbhit())
delay(50); getch();
}
//-----------------------------------int
main() {
         //fullscreen setup bool fullScreen
    = true; int sw =
    GetSystemMetrics(SM_CXSCREEN); int sh =
    GetSystemMetrics(SM_CYSCREEN);
         //initialize graphics window if (fullScreen) { screenW = sw; screenH = sh;
    initwindow(screenW, screenH, "Car Racing - Levels 1 to 11 (Combined)", 0, 0); }
    else {
        initwindow(screenW, screenH, "Car Racing - Levels 1 to 11 (Combined)");
    }
    srand(GetTickCount());
    showIntro(); while
    (gameRunning) {
    showMenu(); }
         //close graphics and
exit closegraph(); return 0; }
```

Results

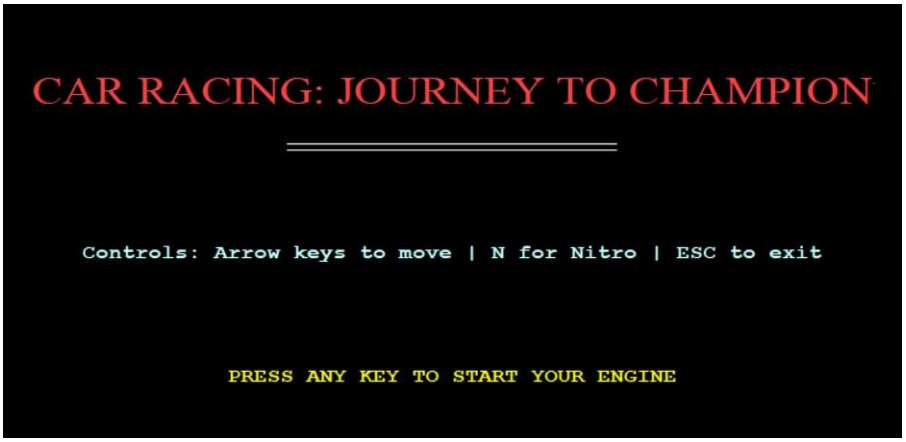


Figure 1 - Introduction Page



Figure 2: Main Menu Page



Figure 3: Select Level Page

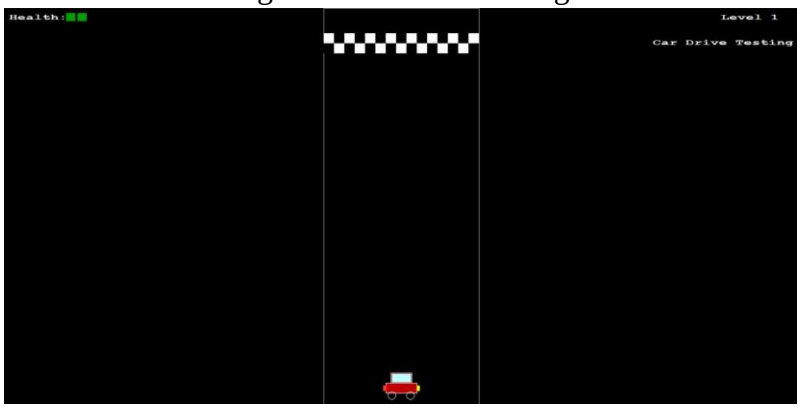


Figure 4: Level 1 - (Car Drive Testing)

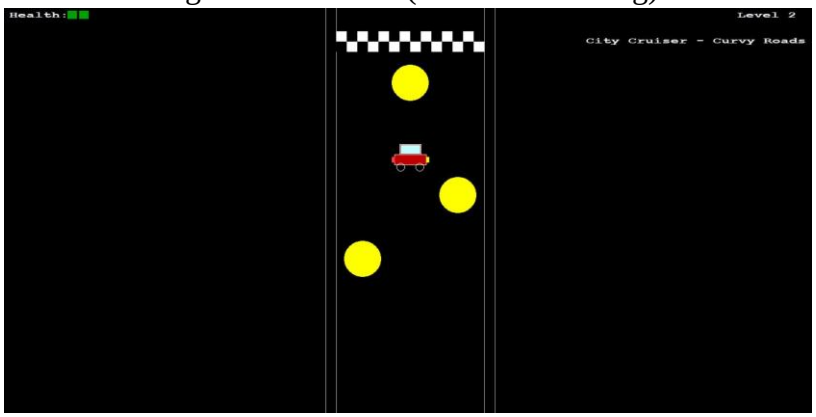


Figure 5: Level 2 (City Cruise - Curvy Roads)

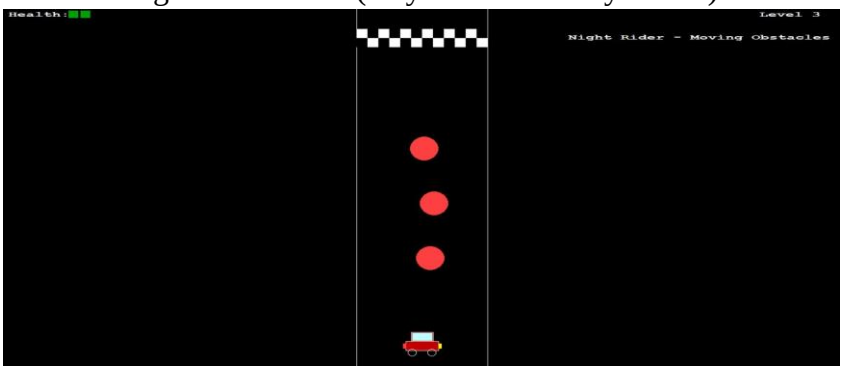


Figure 6: Level 3 (Night Rider - Moving Obstacles)

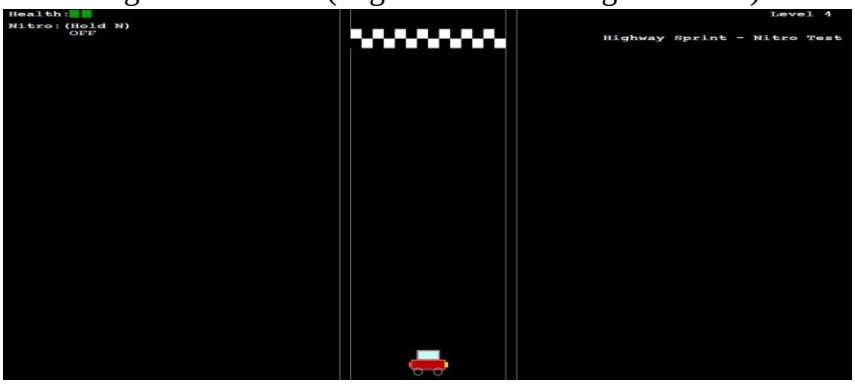


Figure 7: Level 4 (Highway Sprint - Nitro Test)

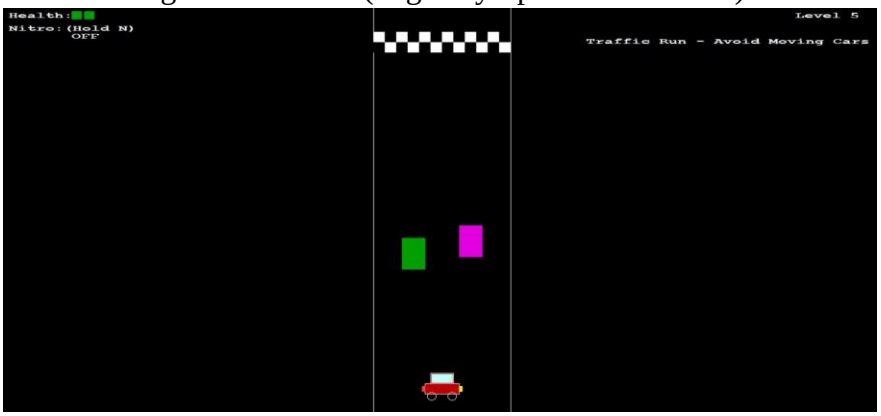


Figure 8: Level 5 (Traffic Run - Avoid Moving Cars)



Figure 9: Level 6 (Rain Rush - Slippery Road)

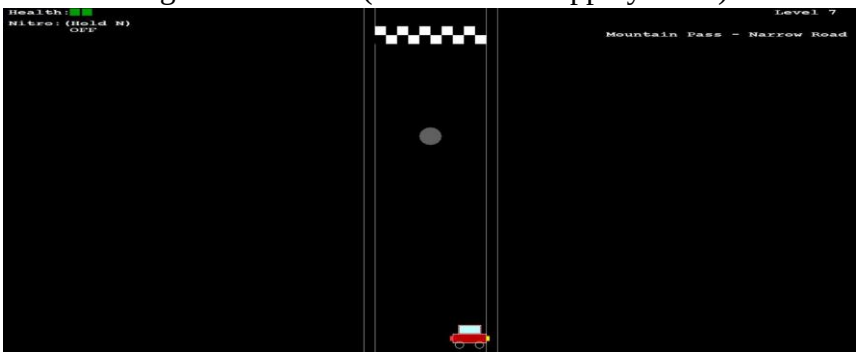


Figure 10: Level 7 (Mountain Pass - Narrow Road)

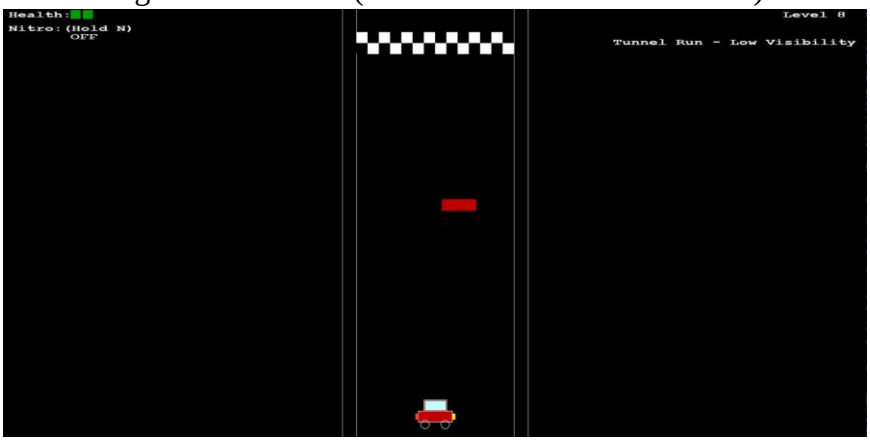


Figure 11: Level 8 (Tunnel Run - Low Visibility)

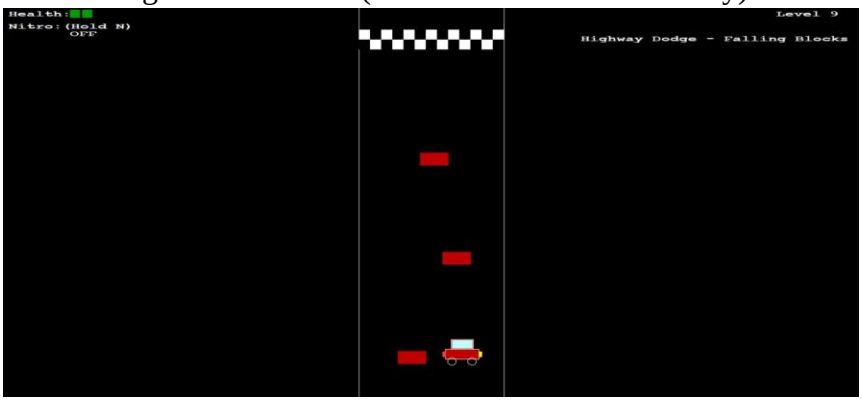


Figure 12: Level 9 (Highway Dodge - Falling Blocks)

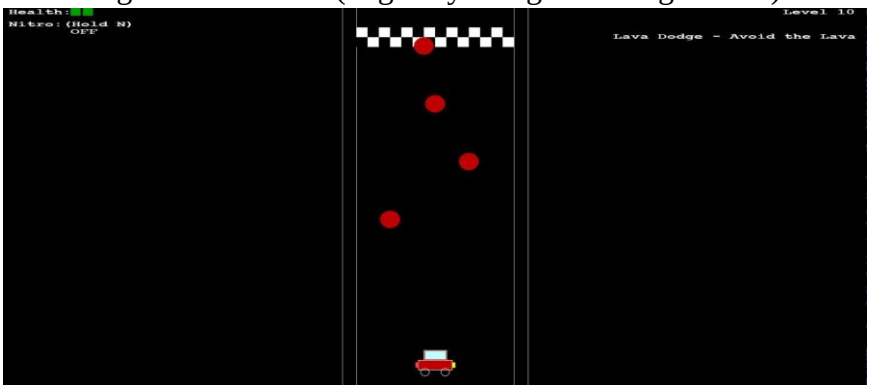


Figure 13: Level 10 (Lava Dodge - Avoid the Lava)

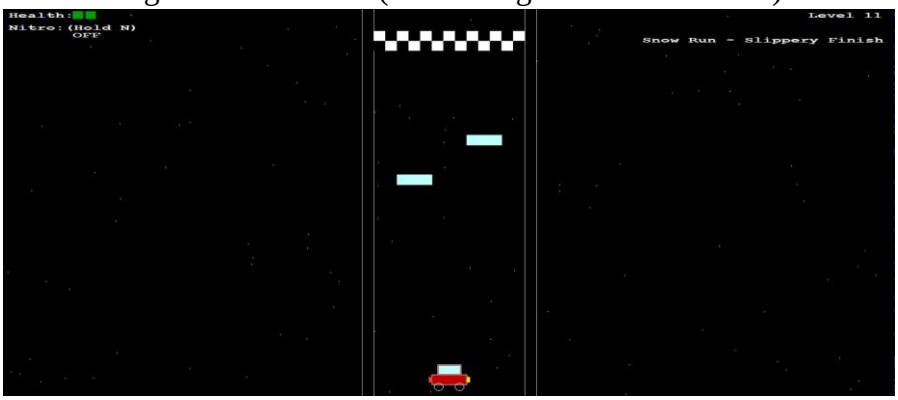


Figure 14: Level 11 (Snow Run - Slippery Roads)



Figure 15: Ending Page



Figure 16: Game Over Page

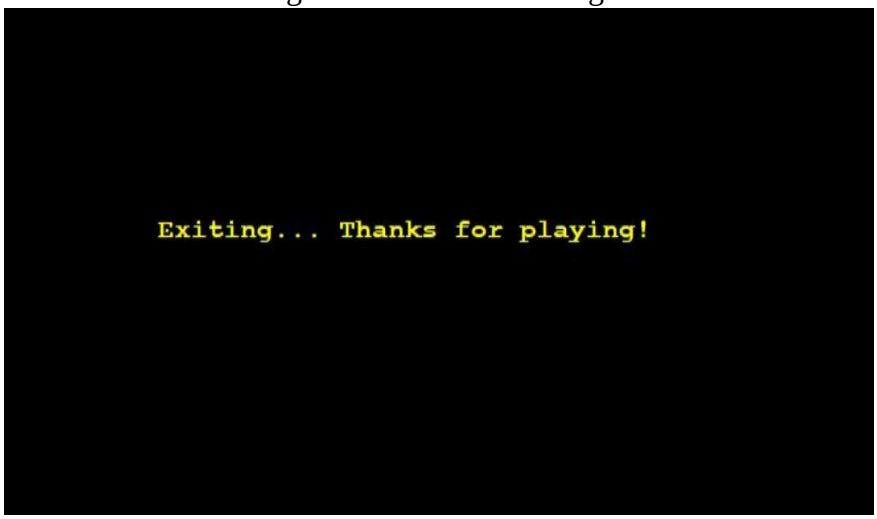


Figure 17: Exiting Page

Challenges Faced and Solutions

a) Inconsistency with frame updates - Causes: flickering and unsmooth movement - Solution: use frame delay ‘delay ()’ and game loop to refresh frames at a consistent rate b) Inaccurate positioning of objects - Solution: i) use consistent coordinates for same codes ii) trial and error with coordinates c) Obstacles moving simultaneously -Solution: uses arrays and loops to control obstacles d) Getting player control responsiveness - Solution: use keyboard with ‘GetAsyncKeyState()’ with delays ‘delay()’ to get responsiveness and stability

Conclusion

To sum it up, the requirements of the Multimedia Programming group assignment are met in the development of the game, which is titled Car Racing: Journey to Champion. With the help of this project we have successfully implemented the concepts of C++ programming to develop a complete and working graphical application which incorporates the visuals/images, logic, and the interaction between the users/program.

We were able to develop a powerful game engine that could accept 11 distinct levels that have different factors in the game such as that of Night Rider (low visibility) and Traffic Run (moving obstacles). The project allowed us to deal with complicated logic, such as:

- Collision Detection: Assisted with mathematical methods, we identified the intersection points of the car of the player (rectangles with the player) and other obstacles (circles and blocks).
- State Management: Global variables and conditional logic were used to manage player health, nitro condition and level advancement in the game.
- User Feedback: There is the inclusion of a Heads-Up Display (HUD) and audio feedback (Beep functions) so that the player is always aware of their game status.

The given assignment was a good practical experience with the graphics.h library, which enhanced our knowledge of the coordinate systems, color filling, and animation loops. Although the task of updating frames and accurate positioning of the objects poses challenges, the end result is a stable game that is entertaining and within the deadline and academic standards that this course required.